

Pipette Commands

Last updated by | Ansse Saarimäki | Sep 25, 2023 at 8:53 AM GMT+3

Pipette Command API

Preparation

1. Connect Pipette with PC (Micro-USB to USB-A)
2. Pipette will automatically register as Serial Interface (COMX)
3. Start Pipette
4. Open Serial Port to pipette or connect via Bluetooth

Pipette Commands

General

- Commands and corresponding responses always end with `\r\n`
- Commands are buffered
- Commands can only be recognized during Protocol mode
- Buttons are recognized everywhere and not buffered
- only motor commands are affected by AUTO flag

Handshake

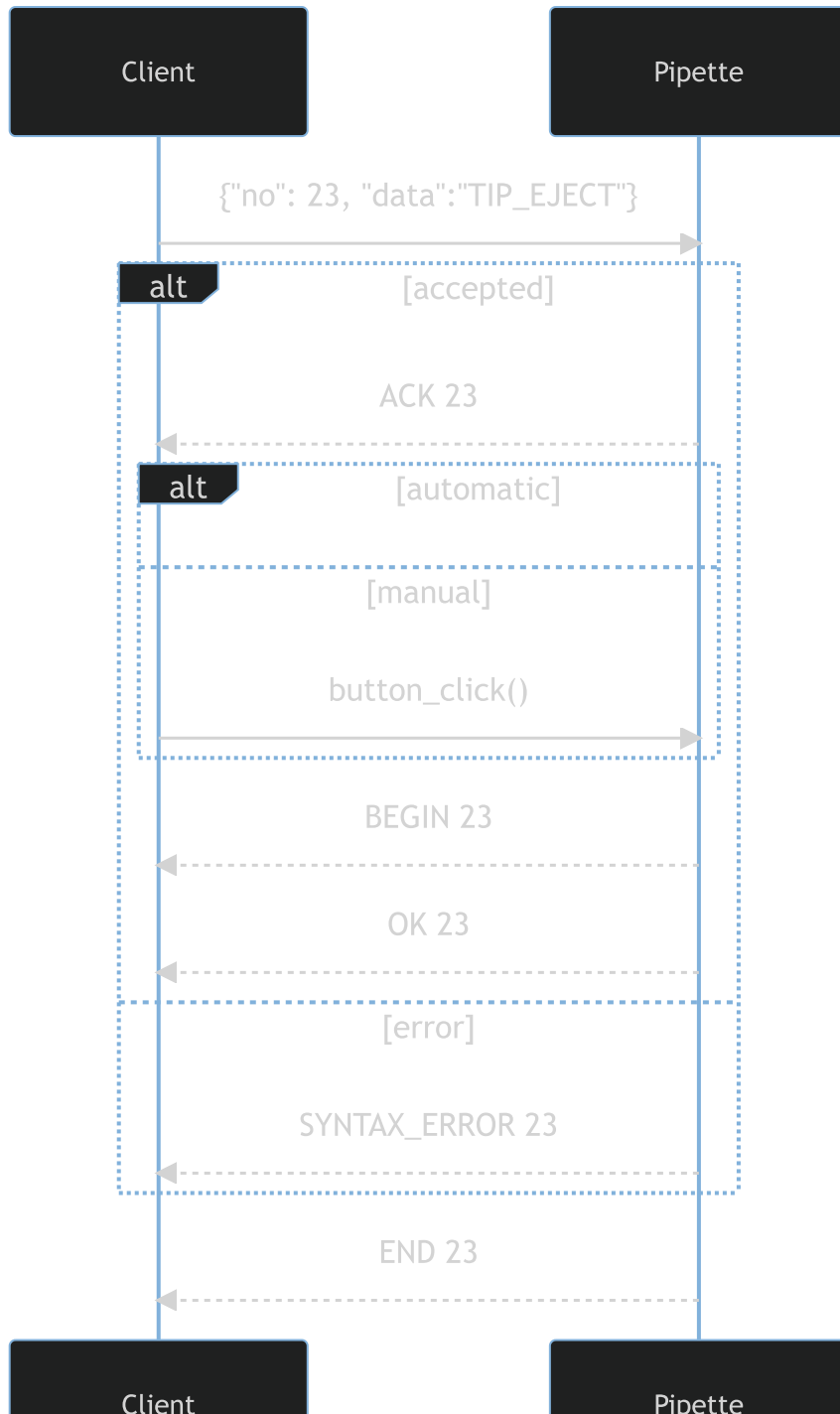
ACK <no>

BEGIN <no> (VERBOSE only)

<**Response**> (only if return value)

<**RESULT**> OK or Error

END <no> (VERBOSE only)



List of button commands

- Picus can be controlled remotely by just emitting button event via the command interface.
- Button commands do not return results

Command	Example	Description
TRIGGER_BUTTON_POWER	<code>{"button": "TRIGGER_BUTTON_POWER"}</code>	Emulates Power Button press.
TRIGGER_BUTTON_LEFT	<code>{"button": "TRIGGER_BUTTON_LEFT"}</code>	Emulates Left (Back) Button press.
TRIGGER_BUTTON_MIDDLE	<code>{"button": "TRIGGER_BUTTON_MIDDLE"}</code>	Emulates Middle (Edit) Button press.
TRIGGER_BUTTON_RIGHT	<code>{"button": "TRIGGER_BUTTON_RIGHT"}</code>	Emulates Right (OK) Button press.
TRIGGER_BUTTON_TOP	<code>{"button": "TRIGGER_BUTTON_TOP"}</code>	Emulates Top (Accept/Run) Button press.
TRIGGER_BUTTON_PRESET	<code>{"button": "TRIGGER_BUTTON_PRESET"}</code>	Emulates Preset (Memory) Button press.
TRIGGER_BUTTON_TIPJECT	<code>{"button": "TRIGGER_BUTTON_TIPJECT"}</code>	Emulates Tip-Eject Button press.
UP	<code>{"button": "UP"}</code>	Emulates Scrollwheel Up movement.
DOWN	<code>{"button": "DOWN"}</code>	Emulates Scrollwheel Down movement.

List of Commands

Command	Parameters	Example	Description	Returns	Buffered
JSON_FORMAT<value>	value: 1 = Response in Json 0 = simple response (Default)	<pre>{"no": 0, "data": "JSON_FORMAT 1"}</pre>	Defines, if the response comes back in Json	Result	no
AUTO<value>	value: 1 = automatic execution 0 = manual execution	<pre>{"no": 0, "data": "AUTO 1", "gui": "AUTO"}</pre>	NoAutomatic command execution	Result	no
VERBOSE <value>	value: 0 = silent mode 1 = verbose mode	<pre>{"no": 0, "data": "VERBOSE 1", "gui": "VERBOS"}</pre>	Switch between verbose and silent handshake mode	Result	no
GET_VERSION	N/A	<pre>{"no": 0, "data": "GET_VERSION"}</pre>	Returns Firmware version (e.g. CP-0.1)	Response	no
SET_BACKLIGHT<0-20>	0 <= <value> >=20	<pre>{"no": 0, "data": "SET_BACKLIGHT 15"}</pre>		Result	no
REBOOT	N/A	<pre>{"no": 0, "data": "REBOOT"}</pre>	Reboot pipette (Standby)	Result	yes

Command	Parameters	Example	Description	Returns	Buffered
RESET	N/A	<code>{"no": 0, "data": "RESET"}</code>	Formats the data flash and reboots pipette	Result	yes
ERASE_DATA <data>	data: USER or FACTORY or PRODUCTION	<code>{"no": 0, "data": "ERASE_DATA USER"}</code>	Erases given data and reboots pipette	Result	yes
SET_BT_MAC <mac>		<code>{"no": 0, "data": "SET_BT_MAC 123456789AFF"}</code>	Set MAC address for Bluetooth	Response	no
SET_SERIAL <serial>	Serial No :<int>	<code>{"no": 0, "data": "SET_SERIAL 1237456"}</code>	Set serial number	Result	no
GET_SERIAL		<code>{"no": 0, "data": "GET_SERIAL"}</code>	Returns serial number	Response	no
GET_ID		<code>{"no": 0, "data": "GET_ID"}</code>	Returns pipette id	Response	no
GET_BATTERY_LEVEL		<code>{"no": 0, "data": "GET_BATTERY_LEVEL"}</code>	Returns battery battery level	Response	no
GET_MIN_VOLUME		<code>{"no": 0, "data": "GET_MIN_VOLUME"}</code>	Returns minimum	Response	no

Command	Parameters	Example	Description	Returns	Buffered
			volume		
GET_NOMINAL_VOLUME		{ "no": 0, "data": "GET_NOMINAL_VOLUME" }	Returns nominal volume	Response	no
GET_MODEL		{ "no": 0, "data": "GET_MODEL" }	Returns pipette model	Response	no
GET_TIME		{ "no": 0, "data": "GET_TIME" }	Returns time	Response	no
GET_DATE		{ "no": 0, "data": "GET_DATE" }	Returns date	Response	no
GET_STATUS_CALIBRATION		{ "no": 0, "data": "GET_STATUS_CALIBRATION" }	Returns calibration status	Response	no
GET_STATUS_MAINTENANCE		{ "no": 0, "data": "GET_STATUS_MAINTENANCE" }	Returns maintenance status	Response	no
GET_STATUS_CHECK		{ "no": 0, "data": "GET_STATUS_CHECK" }	Returns quick check status	Response	no
SET_CAL_DATA <model> <p0> <p1> <p2> <p3> <d0> <d1> <d2> <d3>	model: SINGLE_CHANNEL_10UL, SINGLE_CHANNEL_120UL, SINGLE_CHANNEL_300UL, SINGLE_CHANNEL_1000UL,	{ "no": 0, "data": "SET CAL DATA SINGLE CHANNEL_1000UL 1 2 3 4 5 6 7 8" }	Set calibration points steps	Result	no

Command	Parameters	Example	Description	Returns	Buffered
	SINGLE_CHANNEL_5ML, SINGLE_CHANNEL_10ML, MULTI_CHANNEL_10UL, MULTI_CHANNEL_120UL, MULTI_CHANNEL_300UL, MULTI_CHANNEL_1200UL p/d :<int16_t>				
GET_CAL_DATA <model>	model: SINGLE_CHANNEL_10UL, SINGLE_CHANNEL_120UL, SINGLE_CHANNEL_300UL, SINGLE_CHANNEL_1000UL, SINGLE_CHANNEL_5ML, SINGLE_CHANNEL_10ML, MULTI_CHANNEL_10UL, MULTI_CHANNEL_120UL, MULTI_CHANNEL_300UL, MULTI_CHANNEL_1200UL	<pre>{ "no": 0, "data": "GET_CAL_DATA SINGLE_CHANNEL_1000UL" }</pre>	Get calibration points steps	Response	no
GET_LATEST_LOG		<pre>{ "no": 0, "data": "GET_LATEST_LOG" }</pre>	Returns the log file containing latest entries. Result is FAILED if no logs are available.	Response	no

Command	Parameters	Example	Description	Returns	Buffered
GET_NEXT_LOG		<pre>{"no": 0, "data": "GET_NEXT_LOG"}</pre>	Returns the log file containing next latest entries. Result is FAILED if no more older logs are available.	Response	no
ENABLE_IMPORT_EXPORT <0/1>		<pre>{"no": 0, "data": "ENABLE_IMPORT_EXPORT 1"}</pre>	Enables import/export of pipette data	Result	no
IMPORT <KEY 1*> <PAYLOAD>		<pre>{"no": 0, "data": "IMPORT Settings/Theme", "payload": "VIOLET"}</pre>	Imports Theme settings	Result	no
EXPORT <KEY 2*>		<pre>{"no": 0, "data": "EXPORT Settings/Theme"}</pre>	Gets Theme settings	Result	no
SHOW_SERIAL		<pre>{"no": 0, "data": "SHOW_SERIAL"}</pre>	Shows serial number	Result	no
HIDE_SERIAL		<pre>{"no": 0, "data": "HIDE_SERIAL"}</pre>	Hide serial number	Result	no

Command	Parameters	Example	Description	Returns	Buffered
CLOSE_PROD		{"no": 0, "data": "CLOSE_PROD"}	Clear FactoryMode flag	Result	no
TEST		{"no": 0, "data": "TEST"}	Set TestMode flag	Result	no
CAL_FUEL_GAUGE		{"no": 0, "data": "CAL_FUEL_GAUGE"}	Start fuel gauge calibration. Pipette is reboted at the end of the calibration.	Result	no
ENABLE_MOTOR_CONTROL <0/1>		{"no": 0, "data": "ENABLE_MOTOR_CONTROL 1"}		Result	no
GET_ENCODER		{"no": 0, "data": "GET_ENCODER"}	Returns the absolute encoder value	Response	OK or NOT_ALLOWED
SCREENSHOT	GIF = Returns a Gif with a color palete of 4 colors, GIF_4BIT = Returns a Gif with a color palete of 16 colors, BMP = Returns a Bitmap, BMP_RLE8=	{"no": 0, "data": "SCREENSHOT GIF"}	Returns a Screenshot as Base64 Encoded String	Response	

Command	Parameters	Example	Description	Returns	Buffered
	Returns RLE8 compressed Bitmap, <empty> = Same as GIF				
TOUCH		{"no": 0, "data": "TOUCH"}	Simulate device operation	Response	
CLEAR_MOTOR_CMD_QUEUE		{"no": 0, "data": "CLEAR_MOTOR_CMD_QUEUE"}	Abort current pending motor command and clear motor command queue	Response	
GET_DATE_TIME		{"no": 0, "data": "GET_DATE_TIME"}	Returns the current date and time in ISO format (YYYY-MM-DD hh:mm:ss)	Response	
SET_DATE_TIME <YYYY-MM-DD> <hh:mm:ss>		{"no": 0, "data": "SET_DATE_TIME 2023-05-09 16:00:00"}	Sets the current date and time.	Response	

1*) **KEY** can be one of the following:

- "Reminders/Calibration", "Reminders/Maintenance", "Reminders/QuickCheck"
- "Modes/Diluting", "Modes/Manual", "Modes/MultiAspirating", "Modes/MultiDispensing", "Modes/Pipetting", "Modes/ReversePipetting", "Modes/SeqDispensing", "Modes/Titrating"
- "Settings/Backlight", "Settings/Bluetooth", "Settings/Language", "Settings/Sound", "Settings/Theme", "Settings/TipEjection", "Settings/UserId", "Settings/UserAdjustment", "Settings/UserAdjustmentSlots"
- **NOTE** certain data cannot be imported to the pipette; these include Factory and Production data

2*) **KEY** can be one of the following:

- "Reminders/Calibration", "Reminders/Maintenance", "Reminders/QuickCheck"
- "Modes/Diluting", "Modes/Manual", "Modes/MultiAspirating", "Modes/MultiDispensing", "Modes/Pipetting", "Modes/ReversePipetting", "Modes/SeqDispensing", "Modes/Titrating"
- "Settings/Backlight", "Settings/Bluetooth", "Settings/Language", "Settings/Sound", "Settings/Theme", "Settings/TipEjection", "Settings/UserId", "Settings/UserAdjustment", "Settings/UserAdjustmentSlots"
- "Factory/Adjustment", "Factory/Pipette", "Factory/DurabilityTest"
- "Production/SerialNumber", "Production/MotorLedLevel", "Production/FuelGaugeConfigID", "Production/BluetoothMac", "Production/CalibrationPoints"

List of Motor Commands

In order to be able to execute motor commands, this must first be enabled with the ENABLE_MOTOR_CONTROL command.

Gui tags

Each motor command can have the following "gui_*" -tag, that allows writing a message into the Pipette's screen.

gui_pre: Shown after command is acknowledged / default: is command instruction

gui_while: Shown while command execution / default depends on command

gui_after: Shown after command execution (this will remain) / default : ""

- GUI Commands like WRITE_DISPLAY and WRITE_QR will be overwritten by and gui command
- Example: {"no": 0, "data": "NO_ACTION", "gui_pre": "Hello\nWorld!"}

Some motor commands can have the following "position" tag, that allows to set a position indicator:

```
"position":  
{  
  "title": "Count",  
  "value": "123"  
}
```

The "title" tag allows to set a custom title text up to 5 chars long.

The "value" tag allows to set a custom value text up to 4 chars long.

Command	Parameters	Example	Description	Returns	Buffered	Tracker	Factory Mode only
NO_ACTION	N/A	<pre>{"no": 0, "data":"NO_ACTION", "gui_pre":"NO_ACTION"}</pre>	Ping to check parser readiness	Result	no	no	no
HOME	N/A	<pre>{"no": 0, "data":"HOME", "gui_pre":"HOME"}</pre>	Run to home position	Result	yes	no	no
BLOW_OUT<go_home> <speed> <delay_ms>	go_home: 1/0 speed: 1...9 delay_ms: delay for go_home	<pre>{"no": 0, "data": "BLOW_OUT 1 3000", "gui_pre":"BLOW_OUT"}</pre>	Perform blowout	Result	yes	no	no
RUN_INIT	N/A	<pre>{"no": 0, "data":"RUN_INIT", "gui_pre":"RUN_INIT"}</pre>	Pipette initialization (find home position)	Result	yes	no	no
TIP_EJECT	N/A	<pre>{"no": 0, "data":"TIP EJECT", "gui_pre":"TIP_EJECT"}</pre>	Performs tip ejection	Result	yes	no	
RUN_MIX <cycles> <speed> <amount>	0 <= <amount> <= mixable amount 1 <= <speed> <= 9	<pre>{"no": 0, "data":"RUN MIX 5 7 100", "gui_pre":"RUN MIX", "position":{"title":"Track", "value":"A 1"}}</pre>	Mix specific amount in uL	Result	yes	yes	no
RUN_ASPIRATE <amount> <speed> (IGNORE_MIN_LIMIT)	min volume <= <amount> <= aspiratable amount 1 <= <speed> <= 9 IGNORE_MIN_LIMIT	<pre>{"no": 0, "data":"RUN ASPIRATE 500 7", "gui_pre":"RUN ASPIRATE", "position":{"title":"Count", "value":"123"}}</pre>	Aspirate specific amount in uL	Result	yes	yes	no

Command	Parameters	Example	Description	Returns	Buffered	Tracker	Factory Mode only
	allows volumes smaller that model minimum						
RUN_DISPENSE <amount> <speed> (IGNORE_MIN_LIMIT)	min volume <= <amount> <= dispensible amount 1 <= <speed> <= 9 IGNORE_MIN_LIMIT allows volumes smaller that model minimum	<pre>{ "no": 0, "data": "RUN DISPENSE 1 7 IGNORE MIN LIMIT", "gui pre": "RUN DISPENSE", "position": { "title": "Tube", "value": "X" } }</pre>	Dispense specific amount in uL	Result	yes	yes	no

Command Results

General

Result	Description
OK	Command execution was successful
FULL	Command buffer is full, will be ignored
SYNTAX_ERROR	Command has syntactical errors
ERROR_PARSING	Command payload could not be parsed
UNKNOWN_COMMAND	Command is unknown
MISSING_PARAMETERS	Command is missing input parameters
CHK_ERROR	Firmware data is corrupt
NOT_ALLOWED	Command is currently not allowed
FAILED	Unexpected error
MOTOR_CONTROL_ABORTED	Motor control aborted by user

Motor Errors

Result	Description
INVALID_PARAMETERS	Command parameters are invalid
TIP_EJECT_ERROR	Error in tip ejection process
RUN_TO_ZERO_ERROR	Error in initialization process
UNDERSTEP_OVERSTEP_ERROR	Motor target position could not be reached
FATAL_ERROR	Motor has undefined errors and needs reset